Cathey: An Offline Edge-Deployed Voice Agent for Smart Home Control

Yiwen Chen
yc4653@columbia.edu
Columbia University
New York, New York, USA

Hailin He
hh3185@columbia.edu
Columbia University
New York, New York, USA

Hanzhen Du
hd2592@columbia.edu
Columbia University
New York, New York, USA

Abstract

We present Cathey, a fully offline voice-controlled smart home assistant designed to run entirely on a Raspberry Pi 5 without any cloud dependency. The system classifies spoken commands into four intent categories—*direct_command*, *needs_clarification*, *general_qa*, and *invalid*—and executes corresponding actions on physical GPIO-controlled hardware including an LED ring, stepper motors, and a simulated air conditioner. Cathey combines a rule-based regex fast path (sub-5 ms for unambiguous commands) with a LoRA-fine-tuned Qwen2.5-3B-Instruct model for semantic understanding. A four-layer memory architecture (working, episodic, semantic, and procedural) enables context-aware responses and preference learning. On a 4,000-case evaluation benchmark, the fine-tuned system achieves **94.2%** overall intent classification accuracy, with per-category scores of 96.9%, 90.7%, 89.2%, and 100.0% for the four intent types respectively.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; *Transfer learning*; • **Hardware** → Embedded systems; • **Human-centered computing** → Ubiquitous and mobile computing.

Keywords: voice assistant, edge computing, intent classification, LoRA fine-tuning, smart home, Raspberry Pi, large language models, retrieval-augmented generation

ACM Reference Format:
Yiwen Chen, Hailin He, and Hanzhen Du. 2025. Cathey: An Offline Edge-Deployed Voice Agent for Smart Home Control. In *Proceedings of Advanced Big Data and AI – Final Project (EECS E6895)*. ACM, New York, NY, USA, 6 pages.

1 Introduction

The proliferation of smart home devices has created significant demand for natural language interfaces that allow users to control their environment through speech. Existing commercial solutions—Amazon Alexa, Google Home, Apple HomeKit—rely on continuous internet connectivity, routing every utterance through cloud inference servers. This introduces three practical problems: (1) a latency floor bounded by round-trip network time; (2) a privacy surface where raw audio and transcripts are transmitted to third-party infrastructure; and (3) functional failure whenever connectivity is unavailable.

We address these problems by designing Cathey, a *fully offline* voice-controlled smart home agent deployable on commodity single-board hardware (Raspberry Pi 5, 8 GB). The core challenge is fitting a semantically capable language model within the memory and compute constraints of a 4-core ARM CPU while maintaining acceptable response latency for real-time interaction.

Cathey makes the following contributions:

- A *two-stage inference pipeline* that intercepts approximately 70% of home-control commands with a deterministic regex fast path (<5 ms) and reserves LLM inference for ambiguous and conversational inputs.
- A *quantization benchmark* across five GGUF variants of the Qwen2.5-Instruct family on Pi 5 hardware, identifying 3B-Q3_K_M as the Pareto-optimal configuration (85% accuracy, 3.9 s average latency, 1.5 GB model size).
- A *LoRA fine-tuned model* trained on 22,500 synthesized (utterance, JSON) pairs that improves intent classification accuracy from 85% (base model, Pi benchmark) to 94.2% on a 4,000-case held-out evaluation suite.
- A *four-layer memory architecture* providing working memory, episodic retrieval (ChromaDB), semantic preference storage, and procedural skill learning that eliminates redundant clarification questions for known patterns.
- A *complete hardware integration* driving a WS2812B LED ring, dual 28BYJ-48 stepper motors (curtain and window), and a PWM-controlled fan (AC simulation) through lgpio and pi5neo.

The GitHub repository is available at: https://github.com/Yiwen543/6895-Final-Project-Cathey

EECS E6895, Columbia University, New York, NY 2025.

2 Related Work

Cloud-based voice assistants. Commercial systems such as Amazon Alexa [4] and Google Assistant use cloud-hosted ASR and NLU pipelines, achieving high accuracy but requiring persistent connectivity. Cathey specifically targets the gap left by these systems in offline and privacy-sensitive scenarios.

On-device language models. The emergence of small, instruction-tuned models (TinyLlama [11], Phi-2 [2], Qwen2.5 [7]) has made edge deployment increasingly feasible. The llama.cpp framework [1] enables efficient quantized inference on CPU using GGUF representations. Cathey uses llama.cpp on the Pi and HuggingFace Transformers on GPU-equipped development machines, with backend selection handled transparently at runtime.

On-device speech recognition. Offline ASR on embedded hardware has been made practical by Whisper [8], a transformer-based model trained on 680,000 hours of multilingual audio. The `faster-whisper` library provides CTranslate2-based int8-quantized inference, reducing the `tiny.en` variant (39 M parameters) to under 200 MB memory footprint. Cathey adopts this configuration for its STT front-end, achieving 245–1,065 ms transcription latency on Apple Silicon MPS in end-to-end pipeline tests.

Intent classification for home control. Prior work on home automation NLU includes rule-based slot-filling systems and fine-tuned BERT classifiers [6]. Our approach differs by generating structured JSON directly from an instruction-tuned generative model, unifying slot extraction and intent routing in a single forward pass.

Parameter-efficient fine-tuning. LoRA [3] inserts low-rank adapter matrices into transformer layers, enabling task-specific adaptation with a small fraction of trainable parameters. We apply LoRA with rank $r = 8$, $\alpha = 16$ to Qwen2.5-3B-Instruct, yielding approximately 1.84 M trainable parameters (0.06% of total), trained with TRL SFTTrainer [10] on an H100 GPU.

Retrieval-augmented generation. RAG [5] augments generative models with retrieved context from an external store. Cathey applies RAG at the memory layer: past episodes encoded with `all-MiniLM-L6-v2` [9] are retrieved from a ChromaDB vector store and injected into the QA prompt.

3 System Design

3.1 Architecture Overview

Cathey is organized as a linear pipeline: raw audio → STT → pre-filter → LLM → memory update → TTS → GPIO. Figure 1 shows the module dependency graph.

```
Audio -> STT(Whisper) -> wake_word?
          | no  -> [invalid, 0ms]
          | yes
          v
      try_rule_based()
          | hit  -> GPIO + TTS [<5ms]
          | miss
          v
      LLMParser.parse_unified()
        +- direct_command -> GPIO+TTS
        +- needs_clarif.  -> TTS(ask)
        |    +- reply  -> followup()
        +- general_qa  -> qa() + TTS
          v
      MemoryManager (4 layers)
```

Figure 1. Cathey pipeline. The rule-based fast path intercepts ≈70% of home-control commands before the LLM is invoked.

3.2 Intent Taxonomy

All user inputs are classified into one of four mutually exclusive types:

- **direct_command**: User names a device and an explicit action (e.g., "Cathey, turn on the light," "Cathey, set the AC to 24 degrees"). Produces a structured JSON command with fields `device`, `action`, and `value`.
- **needs_clarification**: User expresses a comfort state without naming a device (e.g., "Cathey, I feel cold," "Cathey, it's too bright"). Triggers a spoken multi-option clarification question.
- **general_qa**: Non-device conversational input including greetings, knowledge questions, and personal statements. Handled by a RAG-augmented QA path.
- **invalid**: Input with no detectable wake word or completely unintelligible content. Suppressed without response.

3.3 Two-Stage Inference

A critical design decision is the *two-stage* approach to inference. First, every transcribed text is checked against a fuzzy wake-word detector (`contains_assistant_name`) using exact substring matching against 9 name variants (Cathey, Cathy, Kathy, Katie, Cathie, etc.) and SequenceMatcher fuzzy comparison with threshold 0.78. Inputs without a detected wake word are immediately classified *invalid* with zero LLM invocation.

Second, wake-word-positive inputs are passed to `try_rule_based()`, which applies a prioritized set of regular expressions over all unambiguous device+action combinations. Matches return sub-5 ms results and skip

the LLM entirely. Only ambiguous, comfort-state, or conversational inputs reach LLM inference.

3.4 Four-Layer Memory

The `MemoryManager` class implements four complementary memory layers:

1. **Working** (RAM deque): Last 8 conversation turns, cleared per session. Used for dialogue continuity.
2. **Episodic** (ChromaDB): Persistent vector store of past (utterance, reply) episodes. Episodes are encoded using `all-MiniLM-L6-v2` (384-d embeddings) and retrieved by cosine similarity (<0.6 distance threshold) for QA prompt injection.
3. **Semantic** (`user_prefs.json`): Structured key-value store for user preferences, e.g., `ac_set_temperature_preference: 22`. Updated automatically after every value-bearing command.
4. **Procedural** (`skills.json`): Maps trigger utterances to resolved actions with a cosine similarity threshold of 0.92. When a new *needs_clarification* input matches a known skill with similarity > 0.92, the stored action is executed directly without prompting the user.

4 Implementation and Data

4.1 Software Stack

The system is implemented in Python 3.10+. Key dependencies are: `faster-whisper` (Whisper tiny.en, CPU int8 STT); `Transformers` + `PEFT` (LLM on GPU/MPS); `llama-cpp-python` (GGUF inference on CPU/Pi); `ChromaDB` (episodic memory); `sentence-transformers` (embeddings); `piper-tts` (neural TTS on Pi); `lgpio` and `pi5neo` (GPIO hardware).

Backend selection is automatic: CUDA and MPS use `transformers` (float16); CPU uses `llama-cpp-python` GGUF Q3_K_M.

4.2 Hardware

The physical system on Raspberry Pi 5 includes: a WS2812B 12-LED ring driven via SPI0 (MOSI = GPIO 10) through `pi5neo`, supporting five color temperature levels (6500K–2700K) and an RGB cycle mode; a 4-pin PWM fan on GPIO 12 (hardware PWM0, 1 kHz) simulating AC output; two 28BYJ-48 stepper motors controlled by ULN2003 drivers—motor 1 (GPIO 5/6/13/26) for the curtain (4,096 half-steps, position tracked 0–100%), and motor 2 (GPIO 17/27/22/23) for the window (2,048 steps, open/close); a USB microphone (16 kHz, mono float32); and a Bluetooth speaker for Piper TTS output via PipeWire (`pw-play`).

4.3 Training Data Synthesis

We synthesized 22,500 labeled (utterance, JSON) training pairs using a template expansion script (`finetune/generate_train_data.py`). The distribution reflects the skew expected in real home-control usage:

- **direct_command**: 7,600 examples — 18 wake-word variants × device-action-value templates covering 10 brightness levels, 5 color temperatures, 15 AC temperatures (16–30°C), 11 curtain positions, and qualitative descriptors.
- **needs_clarification**: 6,500 examples — cold/hot/dark/bright/stuffy comfort phrases × wake-word variants, plus vague atmosphere requests.
- **general_qa**: 6,000 examples — 95 knowledge Q&A pairs × 8 question prefixes × 11 wake-word variants.
- **invalid**: 2,400 examples — no-wake-word commands, filler utterances, and misdirected commands (Alexa, Siri, etc.).

The dataset was split 90/10 into train (20,250 examples) and validation (2,250 examples) sets. Thirteen corrective examples were appended to address identified failure modes (conversational *general_qa* mis-classified as *invalid*; vague temperature requests).

4.4 Fine-Tuning

We applied LoRA ($r = 8$, $\alpha = 16$, dropout 0.05) to all attention and feed-forward projection layers of Qwen2.5-3B-Instruct, yielding 1,843,200 trainable parameters (0.06% of 3.09B total). Training ran for 5 epochs with TRL `SFTTrainer`, batch size 1, gradient accumulation 8 (effective batch size 8), learning rate 2×10^{-4}, cosine schedule with 0.1 warmup ratio, and fp16 precision on an NVIDIA H100 GPU. The final training loss was **0.0913** over 12,660 steps (≈2.9 hours).

Examples were formatted with `tokenizer.apply_chat_template()` using Qwen2.5's native `<|im_start|>/<|im_end|>` format. The system prompt instructs the model to emit exactly one of four JSON schemas corresponding to the four intent types.

4.5 Quantization Benchmark on Pi 5

To select the production model, we benchmarked five GGUF variants of Qwen2.5-Instruct on a Raspberry Pi 5 (ARM64, 4-core Cortex-A76, 8 GB LPDDR4X) with `n_threads=4`, `n_ctx=768`, `temperature=0`, and 3 runs per case (median latency). The test suite

Table 1. GGUF quantization benchmark on Raspberry Pi 5.

Model	Acc	Avg (ms)	P95 (ms)	Size
1.5B-Q3_K_M	15%	5,007	14,051	0.8 GB
1.5B-Q4_K_M	30%	2,728	5,337	1.1 GB
1.5B-Q5_K_M	50%	4,028	6,399	1.3 GB
3B-Q3_K_M	**85%**	**3,887**	**7,581**	**1.5 GB**
3B-Q4_K_M	75%	5,712	9,982	2.0 GB

comprised 20 fixed intent-classification cases (5 per type).

3B-Q3_K_M achieves the highest accuracy (85%) at lower latency than 3B-Q4_K_M (3.9 s vs. 5.7 s, −32%), because fewer bytes per weight reduce memory bandwidth pressure during ARM matrix multiply. We selected this configuration for Pi 5 deployment.

4.6 System Deployment on Raspberry Pi 5

Cathey runs as a Python service on Raspberry Pi 5 OS (64-bit, Debian Bookworm). The main agent loop (`main.py`) is started via `systemd` and brings up the STT, LLM, TTS, and GPIO subsystems at boot. The Bluetooth speaker is registered as the default PipeWire sink via the system Bluetooth daemon. The LoRA adapter is merged into the base model weights on first launch using PEFT's `merge_and_unload()`, which eliminates adapter-dispatch overhead at inference time and reduces per-token latency by approximately 4% compared to the unmerged adapter configuration. GPIO initialization verifies the presence of the `lgpio` daemon and configures all hardware pins before the main loop begins accepting audio input, ensuring clean startup even after a power-cycle reset.

5 Evaluation and Results

5.1 Large-Scale Intent Classification Evaluation

We evaluated the fine-tuned model (Qwen2.5-3B-Instruct + LoRA adapter, running on Apple Silicon MPS) on `evaluation_data.json`, a held-out benchmark of 4,000 cases (1,000 per intent type) constructed from templates entirely disjoint from the training set.

The evaluation runner first applies the wake-word prefilter: inputs without a detected wake word are immediately classified *invalid* without LLM invocation (0 ms). All other inputs are passed to `LLMParser.parse_unified()`, and the predicted `type` field is compared to the ground-truth label.

Latency (LLM calls only, prefilter excluded): p50 = 2,732 ms, p90 = 3,177 ms, p99 = 3,391 ms, max = 4,518 ms.

Table 2. Per-category results (4,000 cases, fine-tuned model).

Intent Type	Acc	Correct	Total	Avg (ms)
direct_command	96.9%	969	1,000	2,656
needs_clarification	90.7%	907	1,000	3,020
general_qa	89.2%	892	1,000	2,332
invalid	100.0%	1,000	1,000	0
Overall	**94.2%**	**3,768**	**4,000**	**2,002**

Table 3. Confusion matrix (row = expected, col = predicted).

Expected \ Got	direct	clarify	qa	invalid
direct_command	**969**	0	0	31
needs_clarification	89	**907**	2	2
general_qa	0	3	**892**	105
invalid	0	0	0	**1,000**

5.2 Confusion Matrix and Error Analysis

Table 3 shows the confusion matrix for all 4,000 cases.

The 232 misclassifications (5.8%) break down as follows:

- **general_qa → invalid (105 cases, 45.3%)**: The dominant failure mode. Conversational fillers ("Cathey, okay thanks," "Kathy, how do I make coffee") fall outside the model's training distribution for short, low-information *general_qa* inputs.
- **needs_clarification → direct_command (89 cases, 38.4%)**: Vague temperature phrases ("raise the temperature," "make it cooler") are over-confidently mapped to `ac/set_temperature`, because the training data contains many explicit temperature commands.
- **direct_command → invalid (31 cases, 13.4%)**: Lexically unusual phrasings ("deactivate the light," "color mode on") that fall outside both the rule-based patterns and the model's learned distribution.

5.3 Software Test Regression Suite

The `software_test.ipynb` notebook contains a 14-case regression suite covering all four intent types plus procedural memory. The rule-based fast path resolved seven direct commands (light on/off, curtain open, window close, AC to 22°C, brightness 50%, RGB cycle) in under 10 ms each. The agent scored **13/14 = 92.9%** on this suite; the single failure was a *general_qa* ("What is your name?") classified as *invalid* by the base model without the LoRA adapter loaded.

The procedural memory test demonstrated correct behavior: on the first occurrence of "Cathey, I feel cold," the system requested clarification; on the second occurrence, the stored preference (window close) was auto-resolved in 5 ms without prompting the user.

5.4 End-to-End Voice Pipeline

Live microphone tests in `software_test.ipynb` (Apple Silicon MPS) demonstrated correct VAD triggering and Whisper STT at 245–1,065 ms per utterance. The wake-word filter correctly rejected all 12 non-Cathey utterances captured in continuous mode (background speech, ambient noise) with zero false positives in the recorded session.

6 Discussion

6.1 Design Insights

The two-stage pipeline (regex fast path + LLM fallback) proved critical for practical usability. The rule-based layer handles the long tail of common, unambiguous commands at deterministic sub-millisecond latency, reducing the frequency of 3–5 s LLM calls from every utterance to only ambiguous or conversational inputs. This architecture is particularly valuable on Pi 5 CPU inference where each LLM call averages 3.9 s.

Injecting device preference state (e.g., `ac_set_temperature_preference`) into the intent classification prompt caused significant degradation: the small model interpreted numeric preference values as evidence of a device-related intent and misclassified *general_qa* inputs. We resolved this by separating concerns: preferences are injected only into the QA answer prompt, never into the classifier prompt. This highlights a general pitfall of context injection with small models: additional context can harm classification accuracy if it shifts the model's prior.

The four-layer memory design follows a separation of concerns that parallels established cognitive memory models. Working memory provides transient dialogue context; episodic memory enables experience-based personalization; semantic memory stores explicit user preferences; and procedural memory encodes learned action-response mappings. In practice, the procedural layer proved most impactful for perceived responsiveness, as it eliminated repeated clarification questions for recurring comfort-state requests within a session. The similarity threshold of 0.92 was chosen empirically to balance recall (avoiding unnecessary re-clarification) against precision (avoiding application of the wrong stored action when an utterance is superficially similar but contextually distinct).

The procedural memory layer (`skills.json`) provides an effective mechanism for preference learning without any retraining. After a single confirmed clarification event, subsequent semantically similar requests ($\geq$0.92 cosine similarity) are resolved automatically, reducing clarification turns and improving perceived responsiveness.

6.2 Limitations

General QA accuracy. At 89.2%, the *general_qa* category is the weakest link. Short conversational inputs ("okay thanks," "never mind") lie in a distribution gap between the *general_qa* and *invalid* training examples. Expanding corrective examples for this boundary would likely recover 3–5 percentage points.

LLM latency on Pi. At 3.9 s average per LLM call on Pi 5 CPU, the assistant is responsive for slow-paced commands but unsuitable for rapid back-and-forth conversation. Larger GGUF quantization (Q4_K_M) improves command accuracy but increases latency to 5.7 s—an unacceptable trade-off for most use cases.

Speech recognition errors. The Whisper tiny.en model, while fast (250–1,065 ms on MPS), produces word errors on uncommon words and model name variants. Several wake-word misses in the continuous loop session were attributable to STT transcription errors, not failures of the fuzzy matcher.

Needs clarification boundary. The boundary between *needs_clarification* ("make it warmer") and *direct_command* ("raise the temperature") is linguistically ambiguous. 89 cases crossed this boundary in the wrong direction. A confidence-thresholded fallback to clarification would address this.

6.3 Future Work

Near-term improvements include: (1) expanding the fine-tuning dataset with augmented negative examples to improve *general_qa* recall; (2) adding a confidence score to LLM output and falling back to clarification when confidence is below a threshold; (3) porting TTS to a lightweight on-device model (e.g., Kokoro) for lower latency than the current Piper neural TTS.

Longer-term directions include personalized LoRA adapters that fine-tune continuously on user feedback, multi-device room-level control, and integration of a retrieval-augmented product manual so the assistant can answer device-specific questions grounded in actual documentation.

7 Conclusion

We presented Cathey, a fully offline smart home voice assistant deployed on Raspberry Pi 5. Through a combination of a deterministic regex fast path, LoRA-fine-tuned Qwen2.5-3B-Instruct, four-layer memory, and direct GPIO hardware integration, the

system achieves 94.2% intent classification accuracy on a 4,000-case benchmark while operating with zero cloud dependency. The quantization benchmark demonstrates that 3B-Q3_K_M is the Pareto-optimal model for Pi 5 deployment, and the two-stage inference architecture makes sub-10 ms response times feasible for the most common smart home commands. Cathey demonstrates that sufficiently capable offline voice agents are achievable on commodity edge hardware with careful co-design of the model, the inference pipeline, and the application layer.

Acknowledgments

The authors thank Prof. Zoran Kostic and the course staff of EECS E6895 Advanced Big Data and AI at Columbia University for project guidance and evaluation framework discussions. Whisper STT is provided by OpenAI [8]. Training compute was provided by Google Colab (H100 GPU). The llama.cpp project [1] and the HuggingFace TRL library [10] were essential to the implementation.

References

[1] Georgi Gerganov. 2023. llama.cpp: Efficient inference of LLaMA models in pure C/C++. https://github.com/ggerganov/llama.cpp.

[2] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. 2023. Textbooks Are All You Need. *arXiv preprint arXiv:2306.11644* (2023).

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.

[4] Veton Kepuska and Gamal Bohouta. 2018. Next-generation of virtual personal assistants (Microsoft Cortana, Apple Siri, Amazon Alexa and Google Home). In *Proceedings of the 2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, 99–103.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9459–9474.

[6] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).

[7] Qwen Team, Alibaba Group. 2025. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2025).

[8] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust Speech Recognition via Large-Scale Weak Supervision. In *International Conference on Machine Learning (ICML)*.

[9] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3982–3992.

[10] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengding Hu. 2020. TRL: Transformer Reinforcement Learning. https://github.com/huggingface/trl.

[11] Peiyuan Zhang, Guangtao Zeng, Tianhao Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. *arXiv preprint arXiv:2401.02385* (2024).